
Contents

Memorie	1
La memoria principale	1
Ordinamento dei byte	1
La memoria e la CPU	2
Memoria cache	3
Memoria secondaria	9
Dischi magnetici	9

Memorie

La memoria principale

La memoria principale(RAM) viene usata per contenere le istruzioni del programma in esecuzione, per la memoria dati e per lo stack/heap delle funzioni.

Questo tipo di memoria è organizzata come insiemi di celle di memoria che possono memorizzare dei dati. Ogni cella è identificata come un indirizzo. Se un calcolatore ha n celle, allora i suoi indirizzi vanno da 0 a $n - 1$ indipendentemente dalla dimensione della cella(normalmente sono di 1 byte oppure talvolta di 4 bit). Il numero di bit nell'indirizzo determina il numero massimo di celle indirizzabili.

Ordinamento dei byte

I processori si distinguono in due categorie che distinguono la posizione del bit più significativo interpretato dal processore:

- big endian, la numerazione va da sinistra a destra, il bit più significativo viene messo all'indirizzo basso della cella
- Little endian, la numerazione va da destra a sinistra, il bit meno significativo viene messo all'indirizzo più basso della cella

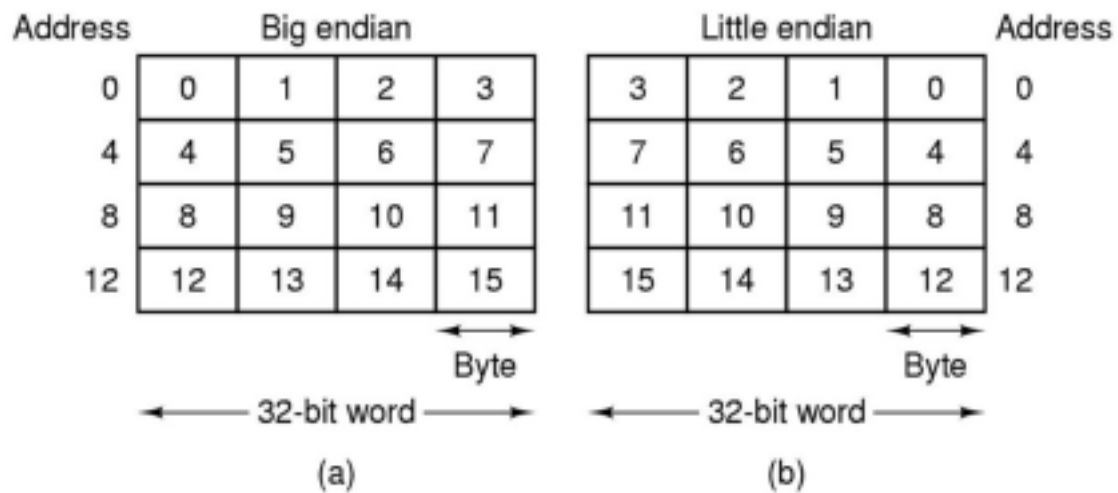


Figure 1: alt text

La memoria e la CPU

Le CPU sono molto più veloci delle memoria, quindi quando la CPU richiede un dato dalla memoria, non la ottiene subito ma aspetta alcuni cicli di clock. Allora è stata inventata una memoria molto veloce ma costosa che fa parte della CPU. Dato il costo sarà molto meno capiente della memoria principale.

Questa è la memoria cache.

Date queste differenze di prezzo e prestazioni, esiste quella che viene chiamata una gerarchia della memoria:

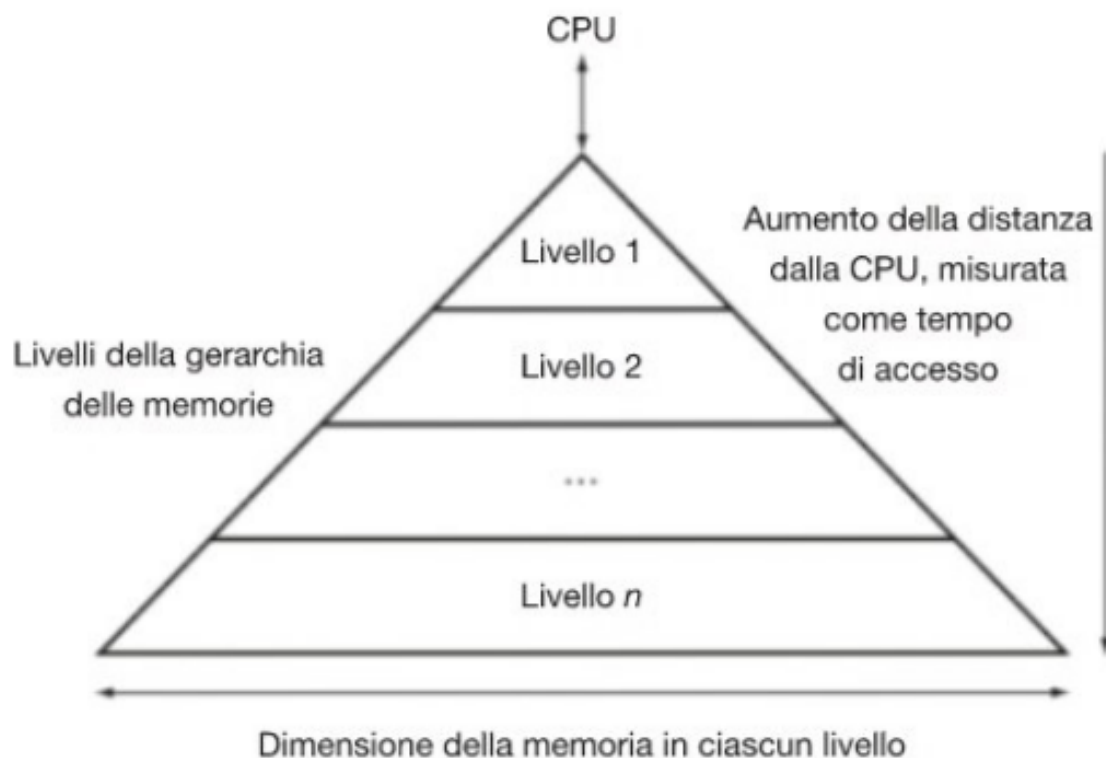


Figure 2: alt text

All'aumentare della distanza dalla CPU, cresce lo spazio.

Memoria cache

La memoria cache è una memoria molto veloce installata vicino alla CPU che è basata sui **principi di località** dei programmi, infatti le celle di memoria che sono più usate nei programmi vengono salvate in questa memoria di ridotte dimensioni.

Il principio di località temporale determina che se si scrive in una determinata posizione di memoria (ad esempio se salviamo un valore nello stack), è molto probabile che dopo qualche istruzione questa cella verrà di nuovo letta, mettendo il valore salvato sia nella memoria sia nella cache riusciamo a ridurre i tempi di accesso in lettura.

Il principio di località spaziale determina che se si accede a determinate celle di memoria, è molto probabile che si accederà alle celle vicine dopo poco tempo. Ad esempio se stiamo iterando un array, è molto probabile che dovremo leggere la cella i -esima ma anche la i -esima+1,...

Inoltre nella cache possono anche essere salvate istruzioni secondo gli stessi principi (ad esempio

se svolgiamo un'operazione molte volte, come nel caso dei cicli, salvare le istruzioni nella cache fa risparmiare tempo alla CPU)

Per mappare indirizzi di memoria principale in una memoria più piccola introduciamo la nozione di **blocchi di memoria**, infatti la memoria principale può essere organizzata in blocchi di k byte che saranno in totale $(M = 2^n/k)$.

La cache è organizzata in linee ed è formata da C linee di k byte dove $(C < M)$, perchè la cache è una memoria più piccola dato il costo).

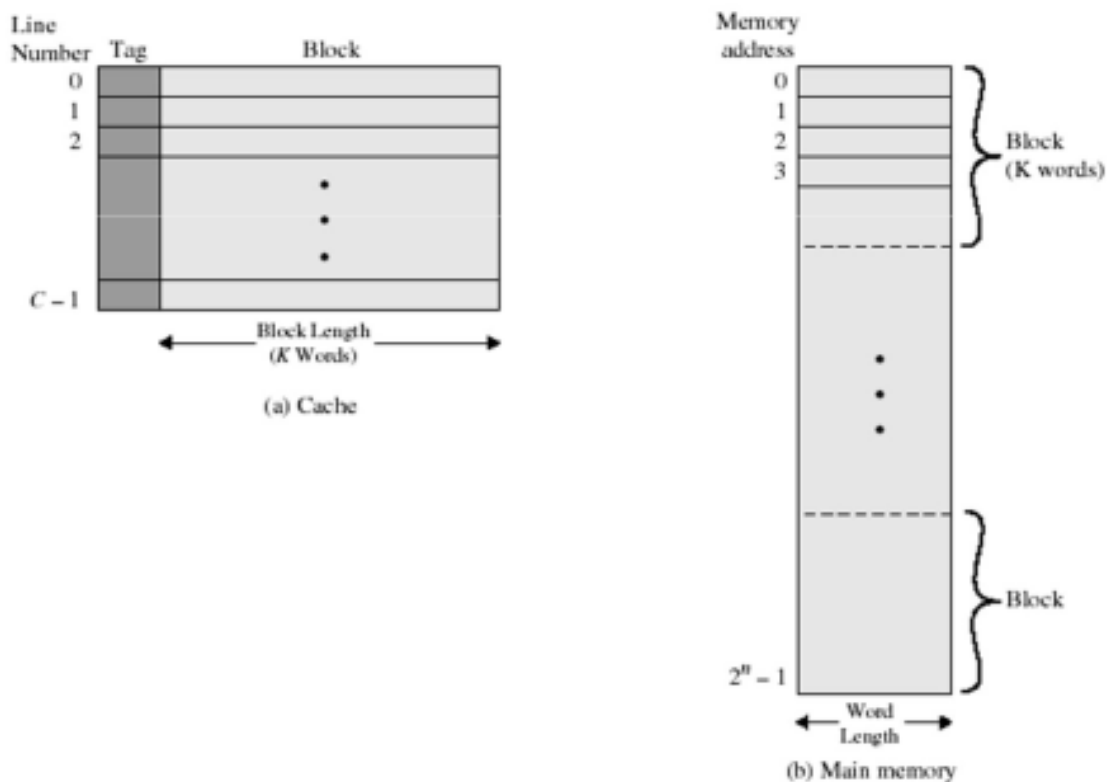


Figure 3: alt text

Se la parola è contenuta nella cache, questa viene inviata alla CPU e avviene un **cache hit**, altrimenti la parola cercata viene caricata nella cache e poi inviata alla CPU (**cache miss**).

Tuttavia le cache hanno diversi problemi di progettazione:

- Dimensione, bisogna ottimizzare la dimensione per contenere il costo

- Numero delle linee, è meglio avere più linee ma di dimensione inferiore o meno linee ma con dimensione superiore?
- Come mappare gli indirizzi della memoria nella cache?
- è meglio separare le cache dati e istruzioni (architettura Harvard) per parallelizzare gli accessi o unificare la cache?
- Quante cache bisogna avere?, oggi siamo arrivati fino a 3 livelli

Mappatura diretta La mappatura diretta (direct mapped cache) permette di mappare un indirizzo in memoria centrale nella cache, questa organizzazione permette di assegnare ogni locazione della cache in modo univoco ad un indirizzo in memoria centrale.

Dato che la cache è una memoria più piccola viene usata la seguente formula per convertire un indirizzo della memoria principale in un indirizzo della cache:

Indirizzo del blocco $\text{mod}(\%)$ numero di blocchi nella cache

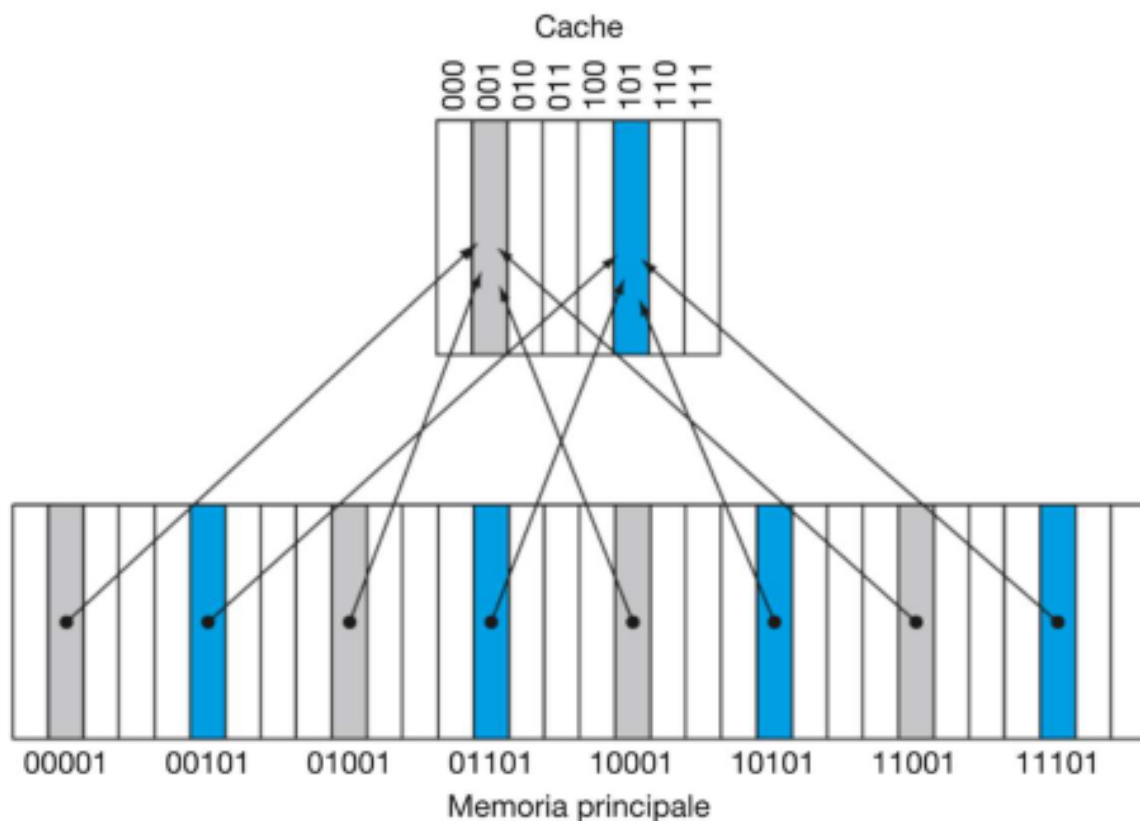


Figure 4: alt text

Indirizzo decimale del dato nella memoria principale	Indirizzo binario del dato nella memoria principale	Hit o miss nell'accesso alla cache	Blocco della cache corrispondente (dove trovare o scrivere il dato)
22	10110 _{due}	Miss (5.9b)	$(10\mathbf{110}_{\text{due}} \bmod 8) = \mathbf{110}_{\text{due}}$
26	11010 _{due}	Miss (5.9c)	$(11\mathbf{010}_{\text{due}} \bmod 8) = \mathbf{010}_{\text{due}}$
22	10110 _{due}	Hit	$(10\mathbf{110}_{\text{due}} \bmod 8) = \mathbf{110}_{\text{due}}$
26	11010 _{due}	Hit	$(11\mathbf{010}_{\text{due}} \bmod 8) = \mathbf{010}_{\text{due}}$
16	10000 _{due}	Miss (5.9d)	$(10\mathbf{000}_{\text{due}} \bmod 8) = \mathbf{000}_{\text{due}}$
3	00011 _{due}	Miss (5.9e)	$(00\mathbf{011}_{\text{due}} \bmod 8) = \mathbf{011}_{\text{due}}$
16	10000 _{due}	Hit	$(10\mathbf{000}_{\text{due}} \bmod 8) = \mathbf{000}_{\text{due}}$
18	10010 _{due}	Miss (5.9f)	$(10\mathbf{010}_{\text{due}} \bmod 8) = \mathbf{010}_{\text{due}}$
16	10000 _{due}	Hit	$(10\mathbf{000}_{\text{due}} \bmod 8) = \mathbf{000}_{\text{due}}$

Figure 5: alt text

Da questa tabella notiamo che per indirizzare la memoria centrale nella cache usiamo i 3 bit meno significativi dell'indirizzo.

Invece per capire se in una linea della cache è presente un dato si usa un bit di validità mentre per controllare se nella linea è presente il valore cercato si usa un insieme di bit detto tag. Il tag contiene la parte superiore dell'indirizzo in memoria centrale in modo che componendolo con l'indirizzo della cache capiamo se c'è il valore cercato oppure no.

Quindi un indirizzo nella cache sarà rappresentato da:

- L'indirizzo ottenuto della cella di memoria
- Il bit di validità che rappresenta se nella linea è presente un dato
- Il tag che indica se è presente il dato cercato

Indice	V	Tag	Dati
000	N		
001	N		
010	S	11 _{due}	Memoria (11010 _{due})
011	N		
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

Indice	V	Tag	Dati
000	S	10 _{due}	Memoria (10000 _{due})
001	N		
010	S	11 _{due}	Memoria (11010 _{due})
011	N		
100	N		
101	N		
110	S	10 _{due}	Memoria (10110 _{due})
111	N		

Figure 6: alt text

Qua abbiamo la tabella completa di una cache.

Possiamo anche calcolare il numero di bit necessari per il campo tag attraverso la seguente formula (data una cache di 2^n blocchi, dimensione del blocco 2^m e i bit degli indirizzi = i): $i - (n + m + 2)$

Il numero totale di bit contenuti in una cache a mappatura diretta è quindi: $2^n \cdot \text{blocksize} + \text{tag size} + 1$

Dove 1 sarebbe il bit di validità.

Dimensione dei blocchi Aumentare la dimensione dei blocchi permette di sfruttare maggiormente la località spaziale e quindi riduce il cache miss, tuttavia se la dimensione dei blocchi diventa troppo grande rispetto alla dimensione della cache salirà nuovamente il miss rate, questo perchè il numero di blocchi che possono essere memorizzati diventa piccolo e cresce la competizione per occuparli, i blocchi vengono scaricati dalla cache ancora prima che vengano utilizzati. Questo perchè quando andiamo ad applicare la formula per calcolare l'indirizzo in cache, se ho un blocco troppo grande vado a riscrivere sempre la stessa cella.

Cache completamente associativa Lo schema di mappatura diretta che abbiamo visto prima permette di scrivere un indirizzo nella memoria centrale in un indirizzo univoco della cache, tuttavia esistono altre tecniche come la cache completamente associativa che permette di scrivere un blocco della memoria centrale in una qualsiasi linea della cache.

Tuttavia questo schema richiede un comparatore parallelo che controlli il tag di tutte le linee della cache aumentando i costi dell'hardware. Questo schema viene usato solo quando si hanno pochi blocchi nella cache per problemi di realizzazione fisica.

Cache set-associativa In una cache set-associativa, una cella di memoria centrale può essere caricata in un numero prefissato di posizioni alternative e una cache a n possibili scelte viene detta cache set-associativa a n vie.

Nelle cache di questo tipo un blocco della memoria principale viene mappato in un'unica linea della cache individuata da un indice, il blocco di dati può essere scritto in qualsiasi linea di questo blocco.



Figure 7: alt text

In questo tipo di cache la linea di cache viene individuata come:

Numero del blocco $\text{mod}(\%)$ numero di linee nella cache

Dato che un dato può essere su qualsiasi blocco della linea, la ricerca va fatta con un comparatore analizzando tutti i blocchi della linea.

Gestione delle miss La gestione della cache miss e cache hit richiede una modifica all'unità di controllo, infatti se avviene un miss nella cache istruzioni avverranno i seguenti passi:

- Inviare il valore originale del PC in memoria
- Leggere dalla memoria centrale l'istruzione contenuta in PC
- Scrivere l'istruzione nella cache calcolando l'indirizzo e il campo tag attraverso l'ALU
- Far ripartire l'esecuzione dell'istruzione dall'inizio ripetendo il fetch che troverà l'istruzione in cache

Anche per la cache dati il procedimento è simile

Gestione della scrittura Le operazioni di scrittura nella cache devono mantenere la coerenza tra la RAM e la cache, infatti se modifichiamo un valore in memoria dobbiamo aggiornare sia la cache, sia la RAM, esistono due tecniche per svolgere quest'operazione:

- Write-through
- Write-back

Write-through Ad ogni scrittura in cache si modifica il valore anche in memoria, se il dato non è presente in cache, viene caricato nella cache poi modificato in cache e infine salvato in memoria. Questo meccanismo non offre buone prestazioni, infatti ogni volta la scrittura di una parola avviene sia in memoria centrale sia in cache. Per mitigare questo problema si può introdurre un buffer di scrittura che permette di aggiornare il dato in cache e quando la CPU ha finito l'istruzione corrente, scrive i dati in memoria centrale. Se il buffer è pieno, la CPU viene messa in stallo finché non si libera spazio nel buffer.

Write-back Il dato viene scritto solo in cache e poi viene aggiornato anche in memoria quando il valore nella cache andrà sovrascritto, questo aggiunge complessità all'implementazione.

Memoria secondaria

La memoria secondaria è la memoria che si occupa dello storage, contiene i programmi che il processore dovrà andare ad eseguire oltre che a file che si possono visualizzare, ecc...

Dischi magnetici

I dischi magnetici sono memorie permanenti meccaniche, sono basate su un gruppo di dischi detti piatti che girano a 5400-15000 RPM, il piatto è rivestito di alluminio magnetizzabile che permette la scrittura/lettura attraverso un braccio mobile contenente una bobina elettromagnetica detta testina.

Dato che ci sono componenti meccaniche in movimento, l'accesso ad una locazione di memoria può avvenire nell'ordine dei millisecondi (ricordiamo che si parla di nanosecondi per i registri).

Ogni piatto è diviso in cerchi concentrici detti tracce, ogni traccia è divisa in settori (512-4096 bytes) che memorizzano le informazioni. In ogni settore è presente un preambolo che serve per sincronizzare la testina, poi ci saranno dei codici di correzione come Hamming o Reed-Salomon e infine un gap tra i vari settori.

Per accedere ai dati il disco deve posizionarsi sulla traccia giusta, questa operazione è detta **seek** e il tempo per spostare la testina sulla traccia viene detta seek time.

Una volta che la testina è nella traccia giusta dovrà aspettare che il settore passi e questo tempo di attesa viene detto latenza di rotazione e questo si può calcolare come:

$$\frac{0.5}{RPM/60}$$

Dove 0.5 è la rotazione media.

Infine l'ultima operazione che andrà fatta sarà il tempo di trasferimento cioè il tempo impiegato a trasferire un blocco di bit, questo è dipendente dalla dimensione del settore, dai giri per minuto e della densità delle tracce. Inoltre i dischi sono equipaggiati da un controller che esegue queste operazioni e può anche contenere una cache per velocizzare questi tempi.

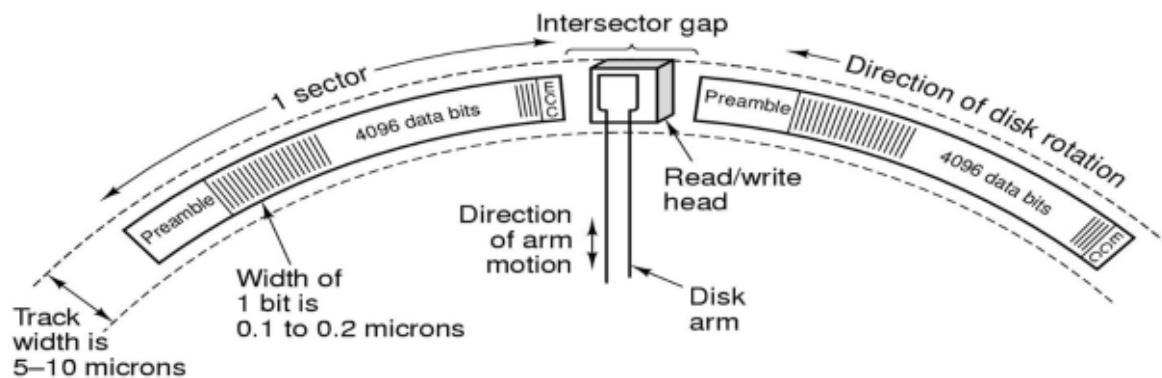


Figure 8: alt text

Esempi di dischi magnetici sono:

- Floppy
- IDE(controller integrato nel disco)
- ATA che aumenta la velocità e l'indirizzamento
- SATA che sfrutta un collegamento seriale per aumentare le velocità e rimpiazzare il connettore

Inoltre i dischi possono essere controllati in due modi:

- SCSI: Interfaccia ad alta velocità che garantisce fino a 7 dispositivi connessi. Permette di collegare hard disk ma anche altre periferiche.
- RAID: Insieme di dischi che possono essere scritti in parallelo. Il sistema operativo lo vede come un disco unico

I dischi magnetici hanno capacità molto grosse(fino a 1TB per piatto) e possono contenere fino a 12 piatti.